

1. **Client and server:** Client and server: a conversation
2. **URLs and DNS:** Anatomy of a URL · DNS: from names to addresses
3. **HTTP messages:** HTTP and HTTPS · Inside an HTTP request · Inside an HTTP response · HTTP methods
4. **Status codes:** Status code families · The status codes you will use
5. **Rendering and DevTools:** From bytes to pixels: how the browser renders · Watching the conversation: the Network tab
6. **The big picture:** Front end and back end · Static and dynamic · From static pages to a full-stack app

Client and server

Client and server: a conversation

The web works as a conversation between two programs: a **client** (usually the browser) sends a **request**, and a **server** answers with a **response**.

- **Request:** what the client asks for (which resource, what to do with it, extra information).
- **Response:** the answer (how it went, plus the content, e.g. the HTML of a page).
- Requests and responses always come in **pairs**, and the client always speaks first.

Example. You open `https://www.cunef.edu/`. (1) The browser sends a request for the page `/`. (2) The server answers with a response that contains HTML. (3) The HTML mentions `styles.css`, `logo.png` and `app.js`, so the browser sends three more requests and gets three more responses. Loading that one page took **four** request/response pairs, and a real site often needs fifty or more.

Common mistake. Thinking the website "lives" in your browser, or that the server keeps sending updates on its own. In plain HTTP the server only ever **answers**. If what you see changes, it is because the browser asked again (you reloaded, or JavaScript requested new data).

URLs and DNS

Anatomy of a URL

A **URL** (Uniform Resource Locator) is the address of one resource: it says how to talk (scheme), to which server (host and port), what to ask for (path), with which options (query string), and where to look inside the result (fragment).

The parts of `https://shop.example.com:8443/products/42?color=red&size=m#reviews`

Part	Value here	What it does
Scheme	<code>https</code>	Protocol to use; <code>https</code> means HTTP over an encrypted connection.
Host	<code>shop.example.com</code>	Name of the server; DNS turns it into an IP address.
Port	<code>8443</code>	Which program on that machine; default 443 for <code>https</code> , 80 for <code>http</code> .
Path	<code>/products/42</code>	Which resource on that server.
Query string	<code>color=red&size=m</code>	Options for the request, as key=value pairs after <code>?</code> .
Fragment	<code>reviews</code>	A position inside the resource; stays in the browser.

Example. In `https://maps.example.com/search?q=madrid&zoom=12#results` the browser contacts `maps.example.com` on port 443 (the default for `https`) and asks for `/search?q=madrid&zoom=12`. Changing `zoom=12` to `zoom=15` asks the same resource with a different option. The `#results` part is never sent: the browser uses it afterwards to scroll to the element with `id="results"`.

Common mistake. Writing a link without the scheme: `<a href="www.example.com">`. Without `https://` the browser treats the value as a **relative path** and looks for a file called `www.example.com` in the current folder, which gives a 404. External links need the full absolute URL.

DNS: from names to addresses

The **DNS (Domain Name System)** is the phone book of the internet: it translates a host name such as `www.cunef.edu` into the **IP address** of the server.

- 1. You type or click a URL.
- 2. **DNS**: the host name is translated into an IP address.
- 3. The browser opens a connection to that IP address and port (for `https`, it also sets up encryption).
- 4. It sends the **HTTP request**.
- 5. It receives the **HTTP response**.
- 6. It **renders** the page, requesting any further files it needs (each one repeats steps 2-5, with DNS answers usually cached).

Example. For `https://www.cunef.edu/grados`, the DNS query asks only for `www.cunef.edu` and returns an IP address. The browser connects to that address on port 443 and sends `GET /grados`, adding a `Host: www.cunef.edu` header, because one IP address can host many different sites and the server needs to know which one you want.

Common mistake. Calling every failure "a 404". If the browser says the server's address could not be found (Chrome shows `DNS_PROBE_FINISHED_NXDOMAIN`), the DNS lookup failed and **no HTTP request was ever sent**, so there is no status code at all. A 404 means the opposite: the server was found and it answered "I do not have that resource".

HTTP messages

HTTP and HTTPS

HTTP (HyperText Transfer Protocol) is the agreed format of the request and response messages between client and server; **HTTPS** is the same protocol sent through an encrypted connection.

Example. HTTP is a postcard: every post office it passes through can read it. HTTPS puts the same postcard in a sealed envelope. If a login form were sent over `http://` on a café network, the email and password would travel as readable text; over `https://` the network only sees that you talked to that server.

Common mistake. Believing the server "knows" it is you because you visited a second ago. Without a cookie or token sent again, your second request is as anonymous to the server as your first one: that is what stateless means.

Inside an HTTP request

An HTTP request is made of a **request line** (method, path, HTTP version), **headers**, an empty line and an optional **body**.

HTTP

```
POST /api/login HTTP/1.1
Host: shop.example.com
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
Accept: application/json
Content-Type: application/json
Content-Length: 48

{"email":"ana@example.com","password":"s3cret!"}
```

Example. Reading the request above line by line: the **request line** says "create something at `/api/login` using `HTTP/1.1`"; `Host` names the site; `Accept` asks for a JSON answer; `Content-Type` announces that the body is JSON and `Content-Length` that it is 48 bytes long; after the **empty line** comes the **body** with the login data. On the server, a framework such as Express parses that body into an object (`req.body`).

Common mistake. Putting secrets in the URL of a GET request, e.g. `/login?password=s3cret!`. URLs are saved in browser history, bookmarks and server logs. Sensitive data belongs in the **body** of a POST request, sent over HTTPS.

Inside an HTTP response

An HTTP response is made of a **status line** (HTTP version, status code, reason phrase), **headers**, an empty line and usually a **body**.

HTTP

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
Content-Length: 131
Cache-Control: max-age=600

<!DOCTYPE html>
<html lang="en">
<head><meta charset="utf-8"><title>Hello</title></head>
<body><h1>Hello, web!</h1></body>
</html>
```

Example. The response above says: it worked (200 OK); the body is HTML encoded as UTF-8; it is 131 bytes long; you may reuse it for 600 seconds. An API answering GET /api/products/42 would send the same structure with Content-Type: application/json and a body such as {"id":42,"name":"Running shoes","price":89.9,"inStock":true}.

Common mistake. Judging the result by the body alone. A server can send a friendly page with status 404, and a badly written API can send 200 OK with {"error":"not found"} in the body. Code that reads a response should check the **status code first**, and a well-designed API treats "200 with an error body" as a bug.

HTTP methods

The **method** (or verb) is the first word of a request and states the intent: **read** (GET), **create or submit** (POST), **replace** (PUT), **partially update** (PATCH) or **remove** (DELETE).

The five everyday methods, on a to-do list API

Method	Intent	Body?	Example
GET	Read a resource or a list	No	GET /tasks/7 returns task 7
POST	Create a resource / submit data	Yes	POST /tasks with the new task creates task 8
PUT	Replace a resource completely	Yes	PUT /tasks/7 with the full new version of task 7
PATCH	Change some fields of a resource	Yes	PATCH /tasks/7 with {"done":true}
DELETE	Remove a resource	Usually no	DELETE /tasks/7

Example. A user ticks task 7 as done. The front end sends PATCH /tasks/7 with the body {"done":true}: only that field changes. If it sent PUT /tasks/7 with {"done":true}, the task would be **replaced** by an object that has no title any more, because PUT means "this is the whole new version".

Common mistake. Using GET for actions that change data, e.g. a link . Browsers prefetch links, and crawlers and chat-app link previews follow GET links automatically, so a bot could delete your data. Changes must use POST, PUT, PATCH or DELETE.

Status codes

Status code families

Every response carries a three-digit **status code**; its first digit gives the family: 1xx informational, 2xx success, 3xx redirection, 4xx client error, 5xx server error.

The five families

Family	Meaning	Who acts next	Typical codes
1xx	Informational: received, still working	Nobody; you rarely see these	100 Continue
2xx	Success	Nobody: use the body	200 OK · 201 Created · 204 No Content
3xx	Redirection or "use your cached copy"	The browser, automatically	301 · 302 · 304
4xx	Client error: bad or forbidden request	The client: fix the request	400 · 401 · 403 · 404

Family	Meaning	Who acts next	Typical codes
5xx	Server error: the server failed	The server's developers	500 · 503

Example. Your page loads but has no styles. In the Network tab, `styles.css` shows **404**: the server is fine, the path in your `<link href="...">` is wrong, so the fix is in your HTML. If it showed **500** instead, the HTML would be fine and the bug would be in the server code.

Common mistake. "4xx means the user typed something wrong." It means the **request** was wrong, and requests are often built by your own code: a misspelled href, a form field missing, a login token not sent. "Client error" names the side of the conversation, not the person.

The status codes you will use

About a dozen codes cover almost every situation; learn them in pairs that are easy to confuse: 200/201, 301/302, 401/403, 404/500.

Common codes

Code	Meaning	Typical situation
200 OK	Success, here is the body	A page or a GET on an API
201 Created	A new resource was created	A successful POST; Location header points to it
204 No Content	Success, nothing to send back	A successful DELETE
301 Moved Permanently	The resource has a new URL for good	A site moved from <code>http://</code> to <code>https://</code>
302 Found	Temporarily at another URL	After logging in, go to the dashboard
304 Not Modified	Your cached copy is still valid	Reloading a page whose CSS did not change
400 Bad Request	The request is malformed or invalid	A required field is missing
401 Unauthorized	Not authenticated	No login token sent, or it expired
403 Forbidden	Authenticated but not allowed	A student opens the admin panel
404 Not Found	No resource at that URL	A misspelled file name in an href
500 Internal Server Error	The server code failed	An uncaught exception in a route
503 Service Unavailable	The server cannot answer right now	Maintenance or overload

Example. A task API: GET `/tasks` without logging in returns **401**; logged in as a student, DELETE `/tasks/3` (a task owned by someone else) returns **403**; GET `/tasks/999` returns **404**; a valid POST `/tasks` returns **201** with Location: `/tasks/8`; a typo in the server code that throws an error returns **500**.

Common mistake. Answering or expecting 500 when a resource does not exist. Nothing failed on the server: the client asked for something that is not there, so the right answer is 404. A 500 should always mean a bug or crash in the server.

Rendering and DevTools

From bytes to pixels: how the browser renders

The browser parses HTML into the **DOM** and CSS into the **CSSOM**, combines them into a **render tree**, computes the **layout** (size and position of every box) and **paints** the pixels; JavaScript can change the DOM, and the browser repeats the steps that are affected.

- 1. **Parse HTML** → **DOM**: elements become nodes of a tree (`html` → `head`, `body` → ...).
- 2. **Parse CSS** → **CSSOM**: all style rules that apply, from the browser defaults and your stylesheets.
- 3. **Render tree**: only the nodes that will be shown, each with its computed styles (an element with `display: none` is in the DOM but not in the render tree).
- 4. **Layout**: the exact size and position of every box for the current window width.
- 5. **Paint**: pixels on the screen.
- When JavaScript changes the DOM, or you resize the window, the browser redoes the affected steps.

Example. A page with `<link rel="stylesheet" href="site.css">` in the `<head>` and a big hero image. The browser builds the DOM, requests `site.css` and the image in parallel, waits for the CSS (it does not want to paint unstyled content), lays out the boxes and paints. When the image arrives, its box gets filled; if the HTML did not give the image a width and height, the layout must be redone and the text below **jumps**.

Common mistake. Thinking JavaScript edits the HTML file. The file on the server never changes; JavaScript edits the **DOM**, the live copy in memory. That is why "View source" (the file as it arrived) and the DevTools Elements tab (the current DOM) can show different things, and why a reload brings the original back.

Watching the conversation: the Network tab

The **Network** tab of the browser DevTools lists every request a page makes, with its method, status code, type, size and time, and shows the headers and body of each one.

Example. Open a news site with the Network tab open and reload. The first row is the document itself (type `document`, status `200`, `Content-Type: text/html`). Then come dozens of rows for CSS, JavaScript, images and fonts, many from other hosts. If your own page shows a red row `logo.png 404`, the Headers panel shows the exact URL that was requested, so you can see which part of your `src` path is wrong.

Common mistake. Opening the Network tab after the page has loaded and finding it empty. The tab only records while DevTools is open: open it first, then reload.

The big picture

Front end and back end

The **front end** is the code that runs in the user's browser (HTML, CSS, JavaScript); the **back end** is the code that runs on the server (for example Node.js with Express) together with the database.

The two halves

	Front end	Back end
Runs on	The user's browser	The server
Typical tools	HTML, CSS, JavaScript, often a framework such as React	A server runtime (e.g. Node.js with Express) and a database (SQL or a document store such as MongoDB)
Good at	Drawing the page, reacting instantly to the user	Storing data, keeping secrets, enforcing rules
Cannot	Be trusted: the user controls it	Change the page directly; it only answers requests

Example. Adding a task in a to-do app: the front end checks that the title is not empty and shows an error immediately (good experience); it then sends `POST /api/tasks`. The back end checks the title **again**, saves it in the database and answers `201 Created` with the new task as JSON; the front end adds it to the list on screen.

Common mistake. Trusting checks done only in the front end. Anyone can open DevTools and delete a required attribute, or skip your page entirely and send the request with `curl`. Every rule that matters (prices, permissions, validation) must be checked again in the back end.

Static and dynamic

A **static** response is a file sent exactly as it is stored, the same for everyone; a **dynamic** response is generated by server code for each request, usually from a database.

Example. A small football club website (home, fixtures, squad, tickets, login) is static: you can open `fixtures.html` straight from your disk (`file:///...`) and it works without any server. A "My tickets" page that lists what the logged-in member bought cannot be static: the server must look up that member in a database and build the answer: that is a job for a back end (see [Server-side JavaScript](#)).

Common mistake. Calling a page "dynamic" because it has animations or a drop-down menu. Those run in the browser. If the server sends the same file to everyone, it is a static site.

From static pages to a full-stack app

A web application is usually built in layers: a **static site** (HTML and CSS), made **interactive** with browser JavaScript, backed by a **server API** that answers with JSON, which stores its data in a **database**, and often topped by a **front-end application** that redraws the page from that data.

- **Front-end developer:** the browser side (HTML, CSS, JavaScript, a front-end framework), plus accessibility and performance.
- **Back-end developer:** the server side (API design, business rules, databases, authentication, security).
- **Full-stack developer:** both halves, and above all the HTTP contract between them: which URLs exist, what JSON they accept and return, and which status codes they answer.
- Whatever the role, the HTTP vocabulary of this section (methods, status codes, headers, JSON bodies, statelessness) is the shared language of every layer.

The layers of a typical web application

Layer	What it adds	Typical tools	Where the code runs
Static site	Content and structure, then presentation and layout	Semantic HTML, CSS (box model, Flexbox, Grid)	Browser
Interactive front end	Behaviour: reacting to clicks, validating forms, changing the page	JavaScript and the DOM	Browser
Server and API	Shared data, rules and secrets, answered as JSON over HTTP	A server runtime and a routing framework (e.g. Node.js and Express), REST design, authentication	Server
Database	Data that survives restarts and is shared by every user	A relational database (SQL) or a document database (e.g. MongoDB)	Server (database server)
Front-end application	One page that fetches data from the API and redraws only what changed	Components, state and client-side routing (e.g. React)	Browser

Example. A task tracker grows layer by layer: a static `tasks.html` with a form; JavaScript that checks the title is not empty and adds the task to the list; an API where `POST /api/tasks` answers `201 Created` and `GET /api/tasks/999` answers `404`; a database so the tasks survive a server restart and every device sees the same list; finally a front-end app that adds the new task to the screen without reloading the page.

Common mistake. Treating "how the web works" as theory you can skip. Every later layer assumes it: server frameworks, API design and authentication all talk about headers, status lines, JSON bodies and statelessness without introducing them again.